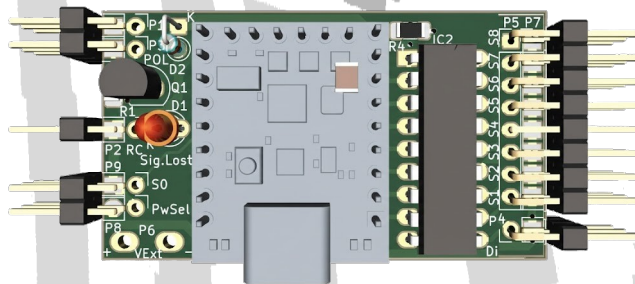


MS8-Xany V3

**Bien plus qu'un simple
MultiSwitch à 8 sorties
pour *OpenAVRc*,
OpenTx, *EdgeTx*,
certains ensembles RC
Futaba et tous les
autres ensembles RC**



Manuel Utilisateur MultiSwitch MS8-Xany V3



Copyright OpenAVRc 2026

Table des matières

1	CE DOCUMENT.....	3
1.1	Versions.....	3
1.2	Copyright.....	3
1.3	Avertissement.....	3
1.4	Contenu.....	4
2	PRESENTATION DE MS8-XANY V3.....	5
2.1	Vue d'ensemble.....	5
2.2	Spécifications du décodeurs MS8-Xany V3.....	6
3	SCHEMA DU DECODEUR MS8-XANY V3.....	8
4	REALISATION DU DECODEUR MS8-XANY V3.....	9
4.1	Circuit imprimé.....	9
4.2	Montage des circuits intégrés sur barrette tulipe.....	9
4.2.1	Montage de l'Arduino Mini Nano V3 sur <i>barrette</i> tulipe.....	9
4.2.2	Montage de l'ULN2803 sur support tulipe.....	9
4.2.3	Montage des connecteurs 3 points en bout de carte.....	10
4.3	Chargement du Firmware dans l'Arduino Mini Nano.....	10
5	UTILISATION/CÂBLAGE.....	11
5.1	Connexion au récepteur.....	11
5.2	Paramètres par défaut.....	11
5.3	Câblage des « utilisations » Tout-Ou-Rien sur les sorties.....	11
5.4	Sélection de la tension d'alimentation des sorties S1 à S8.....	12
5.5	Diodes de roue libre.....	12
5.6	Commande directe de relais avec diode intégrée à l'ULN2803.....	12
5.7	Commande de relais opto-isolés.....	13
5.8	Montage conseillé pour relais 5V opto-isolés.....	14
6	PROTOCOLE ET INTERFACE RC DE MS8-XANY V3.....	15
6.1	Configuration de MS8-Xany V3 pour protocole EKMFA.....	15
6.2	Configuration de MS8-Xany V3 pour protocole Boutons-Poussoirs.....	17
6.2.1	<i>Exemple avec clavier physique (DIY ou Kingpad ou Steuerpad)</i>	18
6.2.2	<i>Calibration de MS8-Xany V3 pour le clavier</i>	19
6.2.3	<i>Comportement des sorties associées aux boutons-poussoirs</i>	20
6.3	Configuration de l'émetteur OpenAVRc pour protocole RCUL/X-Any.....	21
6.4	Configuration de MS8-Xany V3 pour protocole RCUL/X-Any.....	21
6.5	Configuration des émetteurs OpenTx et EdgeTx pour protocole binaire.....	22
6.6	Configuration de MS8-Xany V3 pour OpenTx et EdgeTx pour protocole binaire.....	24
6.7	Configuration de MS8-Xany V3 pour protocole Robbe-Futaba.....	24
7	MODE AVANCÉ/COMMANDE DE SERVOS.....	25
7.1	Utilisation du port série de configuration de MS8-Xany V3.....	25
7.2	L'aide intégrée.....	26
7.3	Les messages de commande de MS8-Xany V3.....	27
7.4	Exemple de configuration réelle.....	31
7.5	Différence entre mode Normal et mode imPulsionnel.....	33
8	LES POSSIBILITES DE DEBUG/DIAGNOSTIC.....	34
8.1	Vérification de la bonne réception du signal RC.....	34
8.2	Vérification de la bonne prise en compte des commandes.....	34
9	COMPENSATION DE L'ERREUR DE DEBIT SERIE.....	35
10	TABLES DES PROTOCOLES ET DES INTERFACES RC SUPPORTÉS.....	36
10.1	Tout ensemble RC.....	36
10.2	Ensemble OpenAVRc.....	36
10.3	Ensembles OpenTx/EdgeTx.....	36
10.4	Ensemble Futaba supportant le MultiSwitch 8 sorties Futaba N° F1513.....	37

1 CE DOCUMENT

1.1 Versions

La version de ce document est la dernière listée dans la colonne **version du document** de la table ci-dessous :

Version du document	Date	Raison de l'évolution
1.0	03/01/2026	Création de MS8 V3 basé sur un Arduino Mini Nano V3
1.1	07/02/2026	Ajout du support des interfaces SRXL, SUMD, IBUS, CRSF
1.2	14/02/2026	Ajout du type Clignotant pour les 8 sorties

1.2 Copyright

Ce document est Copyright © 2023-2026 **OpenAVRc**.

1.3 Avertissement

L'équipe **OpenAVRc** n'est aucunement responsable des dommages qui pourraient découler de la mauvaise utilisation ou d'un éventuel dysfonctionnement de l'émetteur **OpenAVRc**, du module décodeur **MS8-Xany V3** et/ou des logiciels associés.

Il appartient donc à l'utilisateur final d'en mesurer, d'en assumer les risques et de respecter la législation en vigueur selon le pays d'utilisation.

1.4 Contenu

Ce document décrit la réalisation du module décodeur **MS8-Xany V3** ainsi que le paramétrage pour son utilisation avec les émetteurs **OpenAVRc**, **OpenTx**, **EdgeTx** et certains émetteurs **Futaba**. Deux des protocoles supportés (**EKMFA** et **Boutons-Poussoirs**) sont utilisables sur n'importe quel ensemble RC.

MS8-Xany V3 peut se substituer à un décodeur MultiSwitch 8 sorties N° F1513 de **Robbe - Futaba** : il peut donc fonctionner sur un ensemble RC **Robbe-Futaba** qui supporte le décodeur MultiSwitch 8 sorties N° F1513.

Bien que moins pratique, **MS8-Xany V3** peut également être commandé par coups de manche (protocole **EKMFA**), ce protocole fonctionne avec **tous** les ensembles RC.

En ajoutant un clavier avec 8 boutons-poussoirs + résistances, il est également possible d'utiliser **MS8-Xany V3** avec le protocole **Boutons-Poussoirs**. Ce protocole fonctionne avec **tous** les ensembles RC. Le clavier peut être DIY (fait « maison »), un Kingpad de **Pistenking** ou un Steuerpad de **Kraftwerk**.

MS8-Xany V3 est un module décodeur MultiSwitch disposant de 8 sorties « Tout-Ou-Rien ».

En plus de la fonction MultiSwitch classique, **MS8-Xany V3** peut être paramétré afin que ses sorties pilotent des servos en « Tout-Ou-Rien ». Les 2 positions extrêmes ainsi que le temps de déplacement entre ces 2 positions extrêmes sont paramétrables indépendamment pour chaque sortie.

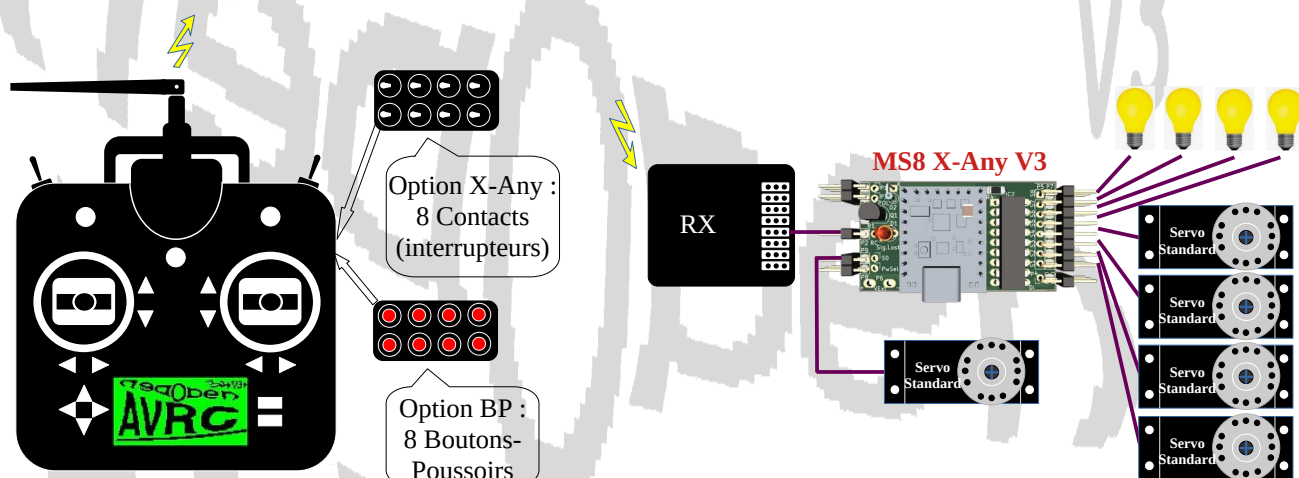
Si piloté à partir d'un émetteur **OpenAVRc**, **MS8-Xany V3** peut fournir une voie proportionnelle d'appoint permettant la gestion d'un servo ou d'un ESC.

Il est également possible de commander chaque sortie en mode « impulsionnel » (équivalent du mode « Memory » chez Robbe-Futaba).

MS8-Xany V3 se paramètre très facilement à l'aide de son interface série USB : un PC et un simple câble USB C (type smartphone) suffisent.

2 PRESENTATION DE MS8-XANY V3

2.1 Vue d'ensemble



Le décodeur **MS8-Xany V3** à 8 sorties se connecte sur le récepteur RC.

L'interface de commande sur le récepteur RC peut être :

- une sortie PWM
- une sortie CPM
- une sortie série SBUS, SRXL, SUMD, IBUS, CRSF

Côté émetteur, la commande peut se faire à partir :

- De coups de manche (Protocole **EKMFA**) : supporté par tous les ensembles RC
- D'appuis sur des boutons-poussoirs (Protocole **Boutons-Poussoirs**) connectés à une voie de l'émetteur
- D'interrupteurs/bouton-poussoirs physiques/logiques (pour les émetteurs évolués : **OpenAVRc**, **OpenTx**, **EdgeTx**, certains ensembles RC **Robbe-Futaba**)

Les protocoles de commande supportés sont :

- **EKMFA** : supporté par **tous** les ensembles RC sans modification
- **Boutons-Poussoirs** : supporté par **tous** les ensembles RC (nécessite l'ajout du « clavier »)
- **RCUL/X-Any** : supporté par les émetteurs **OpenAVRc**
- **Binaire** : supporté par les émetteurs **OpenTx** et **EdgeTx**
- **Futaba** : supporté par certains ensembles RC Robbe-Futaba (Compatibilité avec le MultiSwitch 8 sorties N° F1513)

2.2 Spécifications du décodeurs MS8-Xany V3

Spécification	Valeur / Caractéristique	Note
Alimentation	+4.5V à +6.6V	Mettre le cavalier « tension récepteur » sur « = » ou « ↓ » selon la valeur de la tension fournie par le récepteur
Interface RC PWM	A connecter sur l'entrée RC de MS8-Xany V3	Ne fonctionne qu'avec les protocoles de commande EKMFA, Boutons-Poussoirs, X-Any et Futaba*.
Interface RC CPPM	A connecter sur l'entrée RC de MS8-Xany V3	Ne fonctionne qu'avec les protocoles de commande EKMFA, Boutons-Poussoirs, X-Any et Futaba*.
Interfaces RC séries SBUS, SRXL, SUMD, IBUS, CRSF	A connecter sur l'entrée RC de MS8-Xany V3 Pas besoin d'inverseur SBUS externe	Fonctionnent avec les protocoles de commande EKMFA, Boutons-Poussoirs, X-Any, Binaire (SBUS) et Futaba*.
Protocole de commande X-Any	- Protocole numérique universel utilisé par OpenAVRc pour les accessoires distants - Contrôle d'intégrité par checksum 8 bits - Fonctionne avec tous les protocoles, y compris en 2.4GHz : - Protocole PPM - Protocoles SPIRfMod - Protocoles MultiMod	Contrairement à bon nombre de modules MultiSwitches, MS8-Xany V3 fonctionne également avec les modules HF en 2.4GHz
Protocole de commande Futaba MultiSwitch 8 sorties	Compatibilité avec le module décodeur MultiSwitch 8 sorties N°F1513 de Robbe-Futaba	MS8-Xany V3 peut remplacer avantageusement un décodeur N°F1513 sur un ensemble supportant le décodeur MultiSwitch 8 sorties N°F1513 de Robbe-Futaba.
Protocole de commande « Binaire »	La commande des 8 sorties S1 à S8 est un mot de 8 bits. L'impulsion fournie sur l'entrée RC représente une valeur 8 bits (Valeur de 0 à 255).	Avec ce protocole, il n'est pas possible d'utiliser la sortie proportionnelle S0. OpenTx et EdgeTx doivent utiliser ce protocole.
Protocole de commande « EKMFA »	Ce protocole permet de commander les sorties S1 à S8 par coups de manche	Ce protocole de commande fonctionne sur tous les ensembles RC et fonctionne « au-dessus » des interfaces PWM, CPPM et SBUS.
Protocole de commande «Boutons-Poussoirs»	Ce protocole fonctionne avec un clavier de 8 boutons-poussoirs connecté sur une voie côté émetteur.	Une calibration est à réaliser une seule fois : les largeurs d'impulsion de voie correspondant à l'appui de chaque bouton-poussoir étant sauvegardées en EEPROM.
8 sorties Tout-Ou-Rien	- Commandées en Tout-Ou-Rien par sortie transistorisée	Sorties configurées en mode « numérique » (MultiSwitch)
Alimentation des sorties	- Interne : tension du récepteur - Externe : tension externe 5 à 24V	Sélectionnable par cavalier (commun aux 8 sorties)
Diode de roue libre	Diodes de roue libre des 8 sorties au + de l'alimentation des sorties	Sélectionnable par cavalier (commun aux 8 sorties)
8 sorties Servo	- Commandées en Tout-Ou-Rien - Positions extrêmes paramétrables - Durées des mouvements entre positions extrêmes paramétrables dans les 2 directions	- Sorties configurées en mode «Servo» - Inversion de la course des servos possible en permutant les valeurs extrêmes - Possibilité d'utiliser la tension du récepteur - Possibilité d'utiliser une tension externe (compatible avec les servos)
1 sortie S0 proportionnelle	Commande proportionnelle d'un servo ou d'un ESC de 988µs à 2008µs (0 à 255 pas de 4µs)	Uniquement disponible avec l'émetteur OpenAVRc. Présence de commande proportionnelle détectée en dynamique par MS8-Xany V3 : rien à configurer sauf côté émetteur OpenAVRc
1 sortie analogique (PWM)	Commande d'un PWM possible sur S8, pilotée par la valeur proportionnelle de 0 à 255	Uniquement disponible avec l'émetteur OpenAVRc. Présence de commande proportionnelle détectée en dynamique par MS8-Xany V3 : rien à configurer sauf côté émetteur OpenAVRc
LED rouge perte de signal	Au bout de 1 seconde sans signal	
Failsafe	- Toutes les sorties passent à 0 en cas de perte de signal RC - La sortie proportionnelle S0 conserve sa position courante	Synchrone avec LED rouge perte de signal
Port série USB	Connecteur pour câble USB type C	Pour configuration avancée à l'aide d'une application type « Terminal série »
Dimensions hors tout (mm)	L x l x h : 65 x 28 x 11	

Open

V3

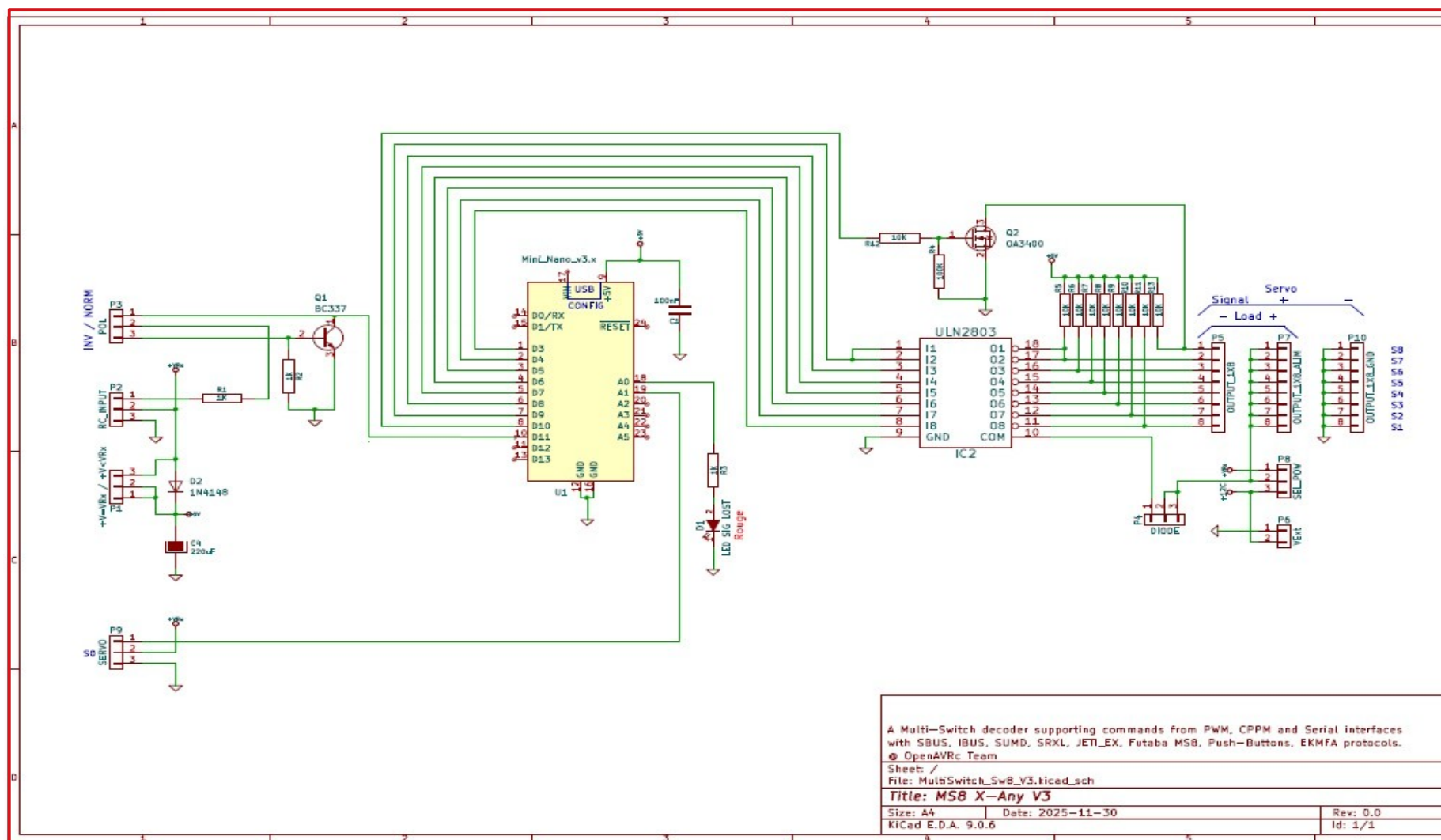
Page left intentionnally blank

/

Page laissée intentionnellement blanche

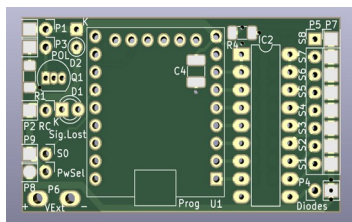
AVRc

3 SCHEMA DU DECODEUR MS8-XANY V3



4 REALISATION DU DECODEUR MS8-XANY V3

4.1 Circuit imprimé

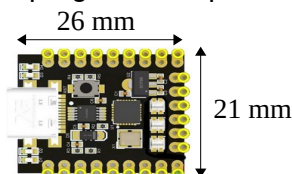


4.2 Montage des circuits intégrés sur barrette tulipe

4.2.1 Montage de l'Arduino Mini Nano V3 sur barrette tulipe

ATTENTION : Il existe 2 versions du Mini Nano V3, prendre celui qui mesure 26 x 21 mm.

L'**Arduino Mini Nano V3** peut être programmé après son positionnement sur le circuit imprimé de **MS8-Xany V3**.



Cependant, il est fortement recommandé de le monter sur 3 portions (2 de 9 points et une de 6 points) de barrette tulipe sécable :



Bien vérifier l'orientation de l'**Arduino Mini Nano V3** avant de mettre sous tension **MS8-Xany V3**.

4.2.2 Montage de l'ULN2803 sur support tulipe

En cas d'erreur de câblage sur les sorties, il est possible de détruire l'**ULN2803**.



Afin de faciliter son éventuel remplacement, il est fortement recommandé de le monter sur un support tulipe de 18 points :

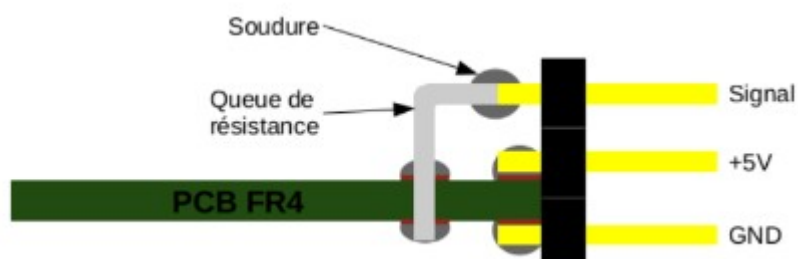


Il est également possible d'utiliser 2 portions (de 9 points) de barrette tulipe sécable :



Dans les 2 cas, bien vérifier l'orientation de l'**ULN2803** avant de mettre sous tension **MS8-Xany V3**.

4.2.3 Montage des connecteurs 3 points en bout de carte



4.3 Chargement du Firmware dans l'Arduino Mini Nano

L'Environnement de Développement Intégré Arduino (IDE) permet de programmer les [Arduino Mini Nano V3](#).

L'[Arduino Mini Nano V3](#) dispose d'un programme préchargé appelé **Bootloader** qui permet, depuis un PC et par USB, le chargement initial puis les éventuelles mises à jour du firmware de [MS8-Xany V3](#).

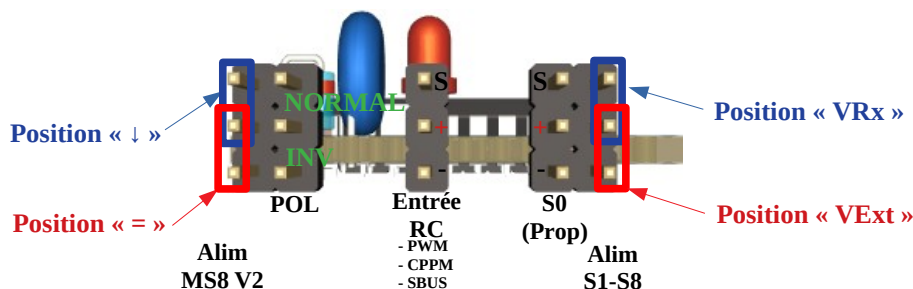
5 UTILISATION/CÂBLAGE

5.1 Connexion au récepteur

Avant de connecter **MS8-Xany V3** au récepteur, il est impératif de mesurer la tension fournie par le récepteur.

Si la tension disponible entre les broches – et + du connecteur 3 points de la voie utilisée est :

1. Supérieure à 5.7V, placer le cavalier « tension récepteur » sur « ↓ »
2. Inférieure à 5.7V, placer le cavalier « tension récepteur » sur « = »



Mettre le cavalier de polarisation POL en position Normal. La position INV est réservée à un usage futur.

5.2 Paramètres par défaut

A la première mise sous tension, **MS8-Xany V3** a les paramètres par défaut suivants :

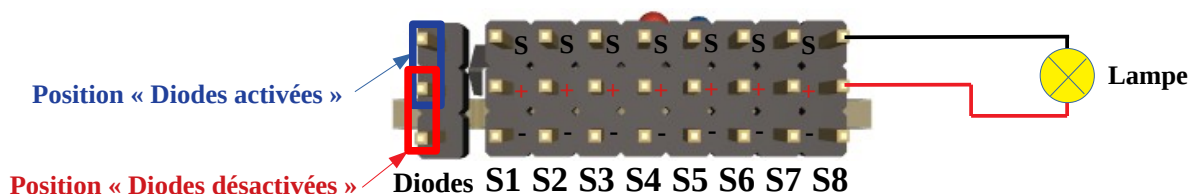
- **I=P** → L'Interface RC de commande est **PWM**
- **P=E** → Le **Protocole** de commande est **EKMFA** (Commande par coups de manche)
- **SR=N** → Le **Servo Reverse** est à **Non** pour la sortie proportionnelle S0
- **Sx=D ; N** → Les Sorties **S1 à S8** sont en type **Digital** en mode **Normal** (Multiswitch)

Pour changer le paramétrage, se reporter au § Utilisation du port série de configuration de MS8-Xany V3.

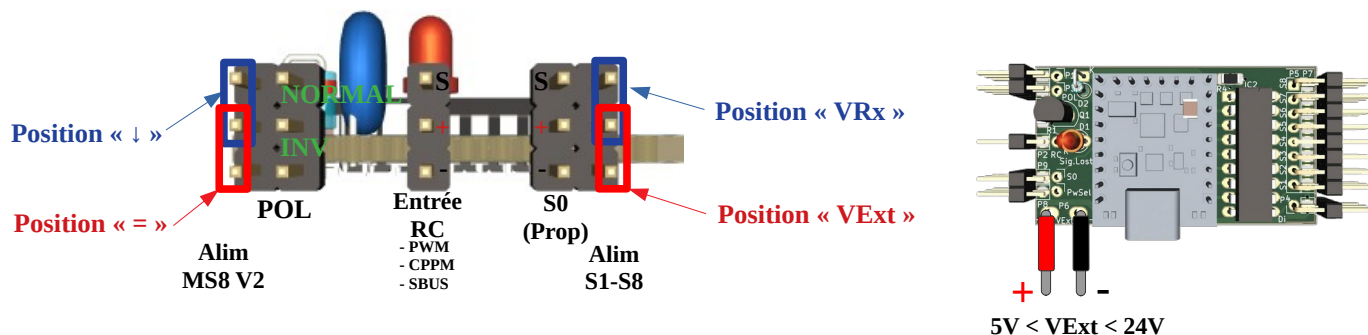
5.3 Câblage des « utilisations » Tout-Ou-Rien sur les sorties

Le **MS8-Xany V3** s'utilise alors comme un module MultiSwitch standard :

- Les « utilisations » (ex : une lampe) se branchent sur les sorties **S1 à S8** entre les broches « S » et « + », la rangée 8 points de « - » en bord de carte n'est pas utilisée dans ce cas.



5.4 Sélection de la tension d'alimentation des sorties S1 à S8

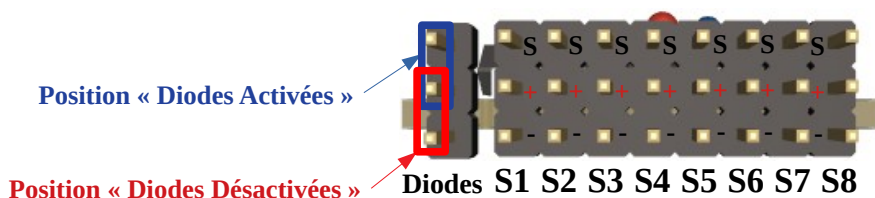


Il est possible d'alimenter les sorties **S1** à **S8** (fourniture du **+**) à partir :

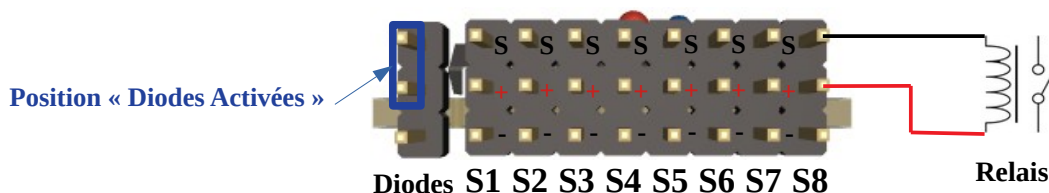
1. De la tension du **+VRx** fournie par le récepteur (Attention à la consommation sur les sorties !)
→ Cavalier de sélection alimentation sur « **VRx** »
2. D'une tension externe **VExt** (de **5V** à **24V**) appliquée sur le connecteur 2 points en bas à gauche de la carte (**ATTENTION** : ne pas dépasser 6V si au moins un servo est connecté à Sx)
→ Cavalier de sélection alimentation sur « **VExt** »

5.5 Diodes de roue libre

Si les « utilisations » connectées sur les sorties sont selfiques (ex :relais), il faut connecter les diodes internes de roue libre, sinon il y a risque de détruire l'ULN2803.



5.6 Commande directe de relais avec diode intégrée à l'ULN2803

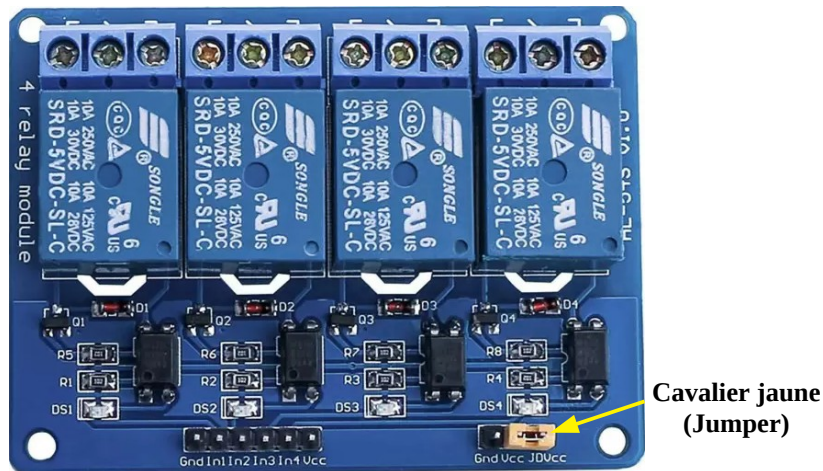


Lorsque des charges selfiques (tels que relais ou petits moteurs) sont alimentés par les sorties S1 à S8, il est nécessaire d'activer les diodes dites « de roue libre ».

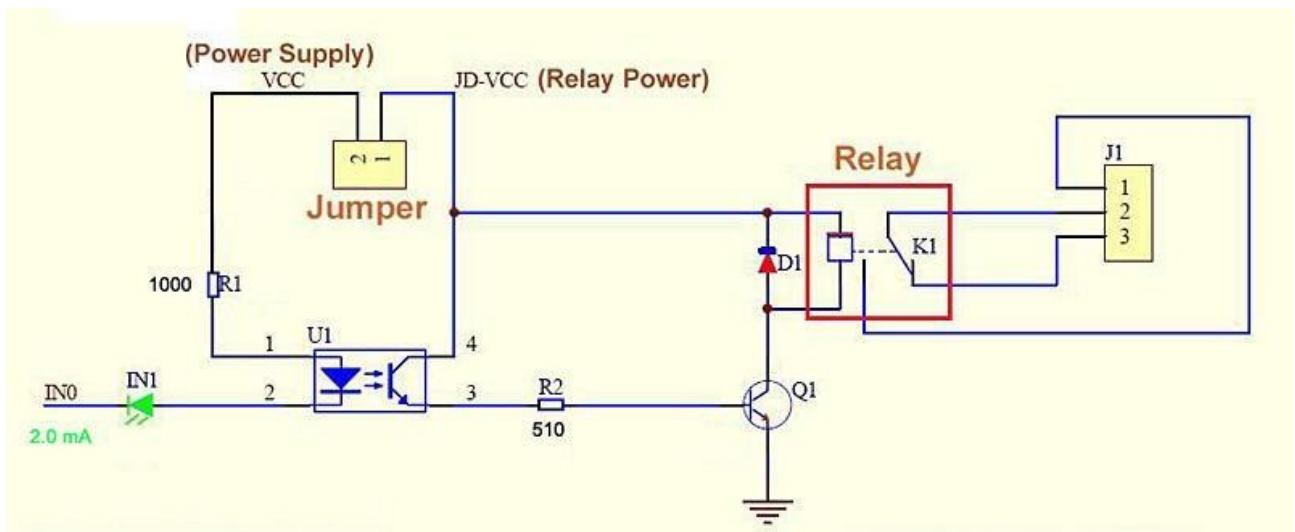
ATTENTION : ces diodes sont activées/désactivées pour les 8 sorties à la fois.

5.7 Commande de relais opto-isolés

Il existe des « modules relais » très bon marché souvent appelés « Arduino Relay Module ». Ces modules intègrent un opto-coupleur permettant une isolation totale entre la tension de commande et la tension d'alimentation des bobines des relais.



Le schéma équivalent d'une voie de ces modules est donné ci-dessous :

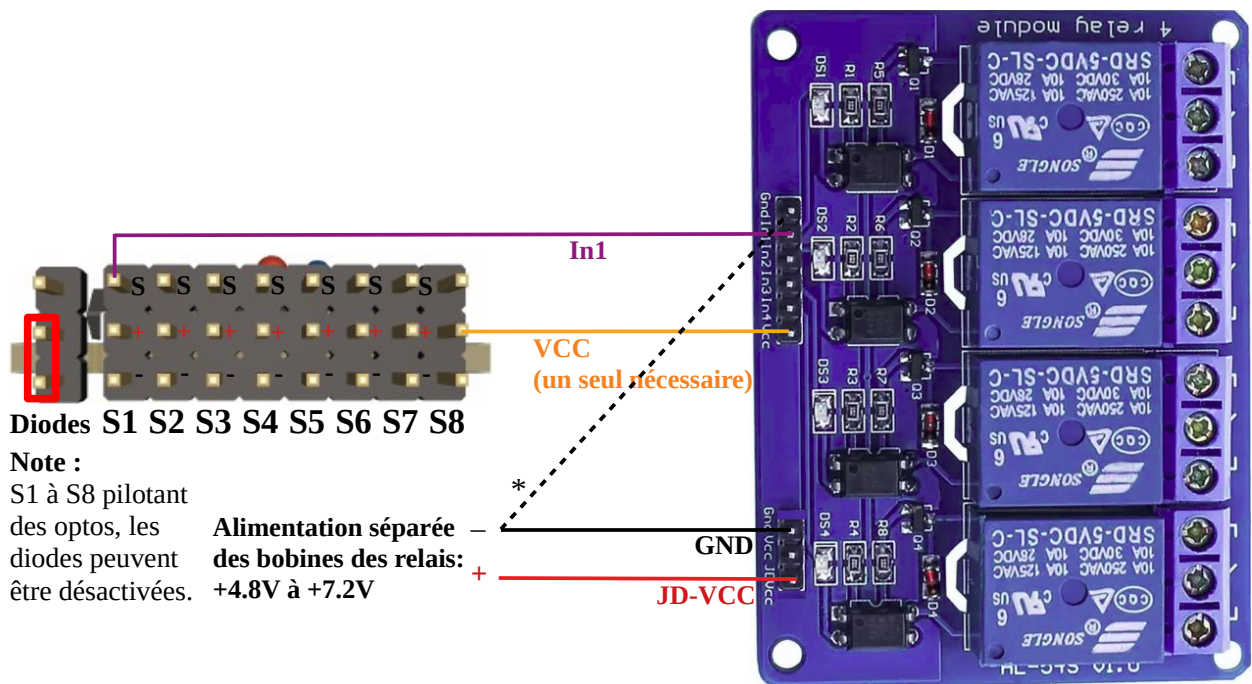


- ◆ **VCC** est la tension de commande des opto-coupleurs : accessible sur le connecteur de bord de carte
- ◆ **JD-VCC** est la tension de commande des bobines des relais : accessible sur un des points du connecteur « Jumper »

Note importante :

Pour bénéficier d'une isolation totale entre VCC et JD-VCC, le Jumper jaune (cavalier) doit être retiré.

5.8 Montage conseillé pour relais 5V opto-isolés



- ◆ Les opto-coupleurs sont alimentés par la tension **VRx** du récepteur (conso : quelques x10 mA)
- ◆ VCC de la carte relais est relié à un des points centraux de S1 à S8
- ◆ In1 de la carte relais est reliée à la sortie S1
- ◆ In2 de la carte relais est reliée à la sortie S2, etc
- ◆ Les bobines des relais sont alimentées par une alimentation séparée (+4.8V à +7.2V)

→ Les alimentations étant complètement isolées, la tension du récepteur ne sera pas perturbée lors des commutations des relais : aucun risque de perdre le contrôle RC.

* : sur certains modèles de « module relais », il n'y a pas de broche **GND** à proximité du **JD-VCC**. Dans ce cas, utiliser la broche **GND** à proximité immédiate de la broche **In1** sur l'autre connecteur.

6 PROTOCOLE ET INTERFACE RC DE MS8-XANY V3

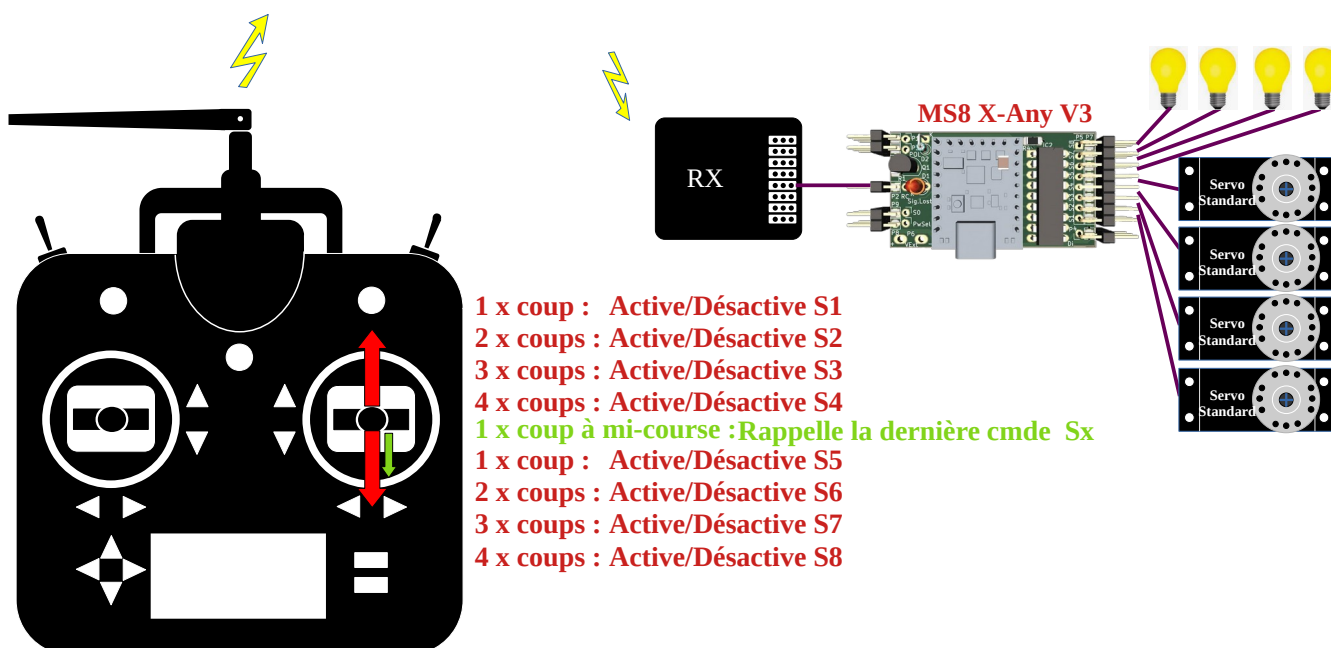
6.1 Configuration de MS8-Xany V3 pour protocole EKMFA

EKMFA signifie **E**in **K**anal **M**ulti-**F**onktion **A**uswahl : un canal pour sélection de fonctions multiples.

Pour commander indépendamment plusieurs fonctions depuis l'émetteur, il suffit de donner des coups de manches. Le nombre de coups de manche, ainsi que la direction des coups de manche sélectionne la sortie à activer/désactiver.

Le comptage du nombre de coups de manche étant fastidieux, les 8 sorties ont été divisées en 2 :

- Un à quatre coups de manche dans **une** direction activent/désactivent les sorties **S1 à S4**
- Un à quatre coups de manche dans **l'autre** direction activent/désactivent les sorties **S5 à S8**



Astuce : un maintien à mi-course vers le bas (flèche verte) pendant environ 1/4 de seconde, complémente la dernière sortie commandée.

Cette manière de commander les sorties est universelle et fonctionne sur tous les ensembles RC et ne nécessite aucune modification de l'émetteur, ni de module additionnel.

De plus, avec **MS8-Xany V3**, côté récepteur, il est possible d'utiliser **EKMFA**, sur une interface RC de type PWM, CPPM ou encore SBUS, SRXL, SUMD, IBUS, CRSF !

PWM et **EKMFA** sont l'interface RC et le protocole par défaut de **MS8-Xany V3**.

A la toute première mise sous tension, **MS8-Xany V3** est déjà dans ce mode.

Si **MS8-Xany V3** a été précédemment configuré dans un autre mode, il est évidemment possible de revenir en **EKMFA** à l'aide des commandes suivantes.

Pour (re-)passer le protocole en **EKMFA** :

- ◆ **P=E** → Le **P**rotocol de commande est **EKMFA**

Les combinaisons d'Interface de commande possibles sont les suivantes :

- ◆ **I=P** → **MS8-Xany V3** se connecte sur une sortie « voie » **PWM** d'un récepteur RC
- ◆ **I=C5** → **MS8-Xany V3** se connecte sur une sortie **CPPM** d'un récepteur RC et utilise la voie N°5 du train **CPPM**
- ◆ **I=S5** → **MS8-Xany V3** se connecte sur une sortie **SBUS** d'un récepteur RC et utilise la voie N°5 de la trame **SBUS**
- ◆ **I=X5** → **MS8-Xany V3** se connecte sur une sortie **SRXL** d'un récepteur RC et utilise la voie N°5 de la trame **SRXL**
- ◆ **I=D5** → **MS8-Xany V3** se connecte sur une sortie **SUMD** d'un récepteur RC et utilise la voie N°5 de la trame **SUMD**
- ◆ **I=I5** → **MS8-Xany V3** se connecte sur une sortie **IBUS** d'un récepteur RC et utilise la voie N°5 de la trame **IBUS**
- ◆ **I=F5** → **MS8-Xany V3** se connecte sur une sortie **CRSF** d'un récepteur RC et utilise la voie N°5 de la trame **CRSF**

Note :

Pour les interfaces séries **SBUS**, **SRXL**, **SUMD**, **IBUS** et **CRSF**, il peut être nécessaire d'ajouter le paramètre optionnel **;-** ou **;** à la fin de la commande (cf §COMPENSATION DE L'ERREUR DE DEBIT SERIE).

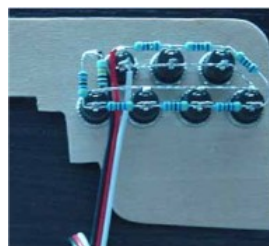
Ex. **I=S5; +**

6.2 Configuration de MS8-Xany V3 pour protocole Boutons-Poussoirs

Dans ce mode de fonctionnement, côté émetteur, un « clavier » comprenant au minimum 8 boutons-poussoirs se connecte sur une voie libre ou se substitue à un potentiomètre de manche.

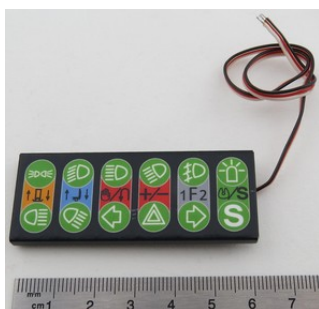
Le clavier peut être de type :

- Clavier DIY (fait maison) avec assemblage de 8 boutons-poussoirs et de résistances :



Exemple : clavier DIY à 7 boutons-poussoirs (il en faut 8 pour **MS8-Xany V3**)

- Clavier **Kingpad** de **Pistenking** ou **Steuerpad** de **Kraftwerk** :



Clavier **Kingpad**



Clavier **Steuerpad**

- Clavier virtuel tactile (script lua) pour émetteurs **Ethos** de **FrSky** (X20 ou X20S) :

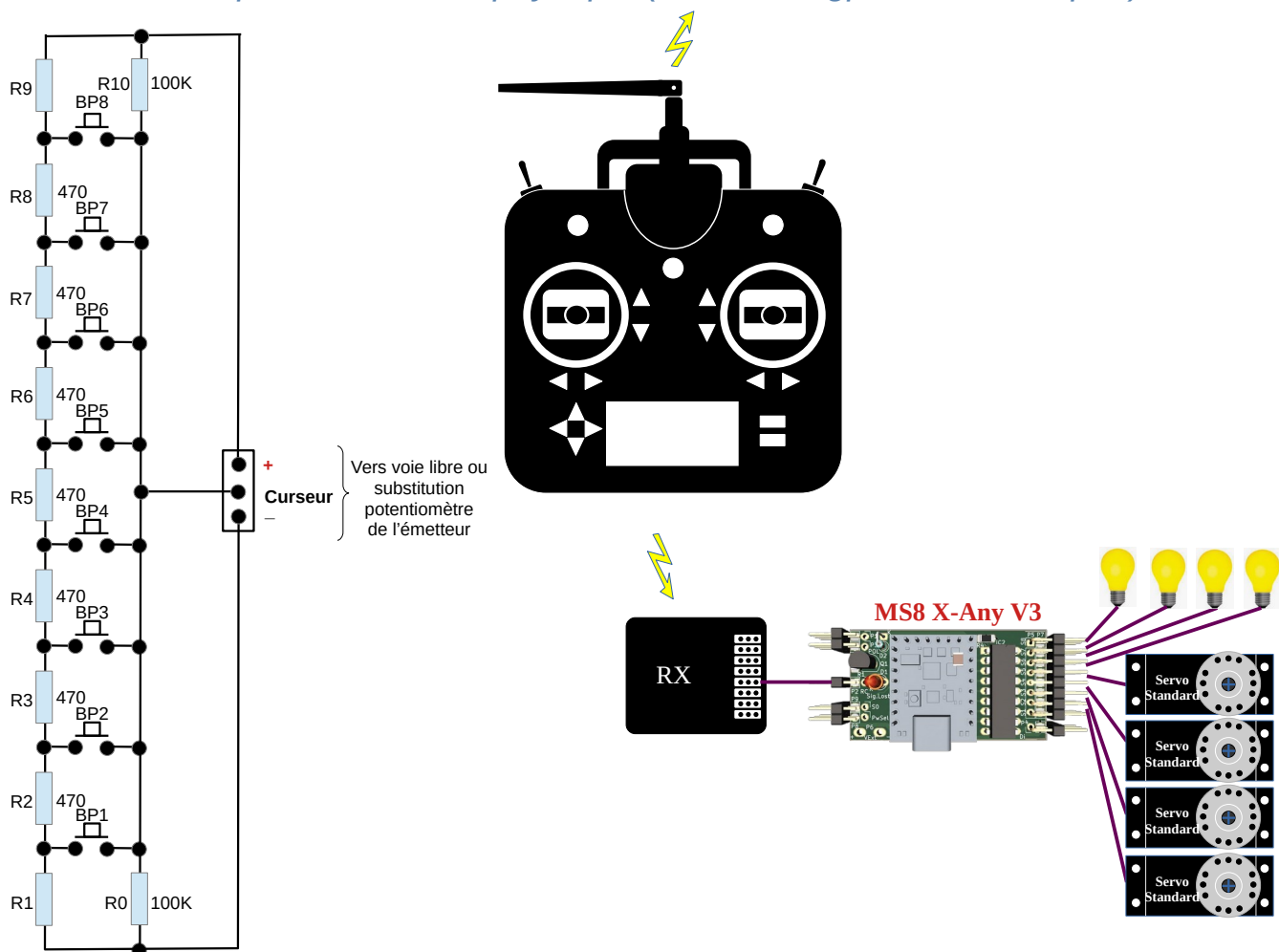


Script lua **Smart Kingpad** pour **X20** de **FrSky**

Note :

MS8-Xany V3 n'utilise que 8 des 12 touches du **Kingpad** ou du **Steuerpad**.

6.2.1 Exemple avec clavier physique (DIY ou Kingpad ou Steuerpad)



Clavier DIY 8 touches

Valeurs de R1 et R9 :

Malheureusement, chaque fabricant d'ensemble RC a conçu son propre étage analogique pour les potentiomètres dans les émetteurs : il n'y a aucune standardisation. De plus, il peut y avoir des différences entre 2 modèles d'émetteurs d'un même fabricant ! C'est pourquoi la valeur des 2 résistances extrêmes R1 et R9 (qui sont identiques) doivent être adaptées selon l'émetteur.

En général, les fabricants utilisent des potentiomètres ayant une course de 270°, mais ces derniers ne sont utilisés que sur 90° (course réelle des manches), soit le tiers de la course totale.

Il faut donc que R1 et R9 soient égales à la somme des résistances R2 à R8. Une valeur de 3.3K (7 x 470) pour R1 et R9 devrait donc convenir dans la majorité des cas.

Chaque appui sur un bouton-poussoir génère une tension différente au niveau du point noté « curseur », l'émetteur va donc générer une impulsion de largeur différente qui va se retrouver côté réception sur la voie associée.

Bien que chaque sortie de **MS8-Xany V3** puisse être commandée indépendamment des autres, les appuis multiples sur le clavier ne sont pas supportés et ne conduiront pas aux commandes attendues !

La commande D=1 permet de vérifier la largeur d'impulsion associée à chaque bouton-poussoir.

6.2.2 Calibration de MS8-Xany V3 pour le clavier

Cette procédure est valable quel que soit le type de clavier (physique ou virtuel).

Pour passer le protocole en mode **Boutons-Poussoirs**, envoyer la commande suivante :

- ◆ **P=P** → Le **P**rotocol de commande est Boutons-**P**oussoirs

Les combinaisons d'Interface de commande possibles sont les suivantes :

- ◆ **I=P** → **MS8-Xany V3** se connecte sur une sortie « voie » **PWM** d'un récepteur RC
- ◆ **I=C3** → **MS8-Xany V3** se connecte sur une sortie **CPPM** d'un récepteur RC et utilise la voie N°3 du train **CPPM**
- ◆ **I=S3** → **MS8-Xany V3** se connecte sur une sortie **SBUS** d'un récepteur RC et utilise la voie N°3 de la trame **SBUS**

Calibration en mode Boutons-Poussoirs

MS8-Xany V3 doit connaître la largeur d'impulsion associée à chaque appui sur les boutons-poussoirs BP1 à BP8 : il y a donc une phase de calibration initiale à réaliser (une seule fois).

Pour réaliser cette opération, il faut connecter un Terminal Série au décodeur **MS8-Xany V3**.

Pour passer en mode calibration pour les boutons-poussoirs, envoyer la commande suivante :

- ◆ **B=C** → Passe le protocole **B**outons-Poussoirs en mode **C**alibration

L'affichage devient :

BP1 :

Tout en maintenant le Bouton-Poussoir N°1 appuyé, appuyer sur la touche Entrée du clavier du Terminal série.

La largeur d'impulsion mesurée pour le BP1 va alors être affichée (et automatiquement mémorisée) et BP2 va être affiché :

BP1 : 0988

BP2 :

Tout en maintenant le Bouton-Poussoir N°2 appuyé, appuyer sur la touche Entrée du clavier du Terminal série.

Répéter l'opération jusqu'au BP8.

Une fois la largeur d'impulsion associée au BP8 affichée, la calibration est terminée.

Il est possible d'afficher les 8 largeurs d'impulsion associées aux BP1 à BP8 par la commande :

B?

B=0988;1100;1228;1320;1672;1768;1888;2016

En cas de fausse manipulation ou d'erreur, il est évidemment possible de refaire une calibration en renvoyant la commande :

- ◆ **B=C** → Passe le protocole **B**outons-Poussoirs en mode **C**alibration

Note : Il doit y avoir au minimum 80 µs d'écart entre 2 largeurs d'impulsions adjacentes (sinon revoir la valeur de R1 et R9 du clavier à bouton-poussoirs).

6.2.3 Comportement des sorties associées aux boutons-poussoirs

1. Si la sortie est configurée en mode **Normal**, elle est active tant que le bouton-poussoir associé est appuyé et se désactive dès que l'on relâche le bouton-poussoir.

Ex :

- Pour une sortie numérique : **S1=D; N**
- Pour une sortie clignotant: **S1=B; N; 250; 750**
- Pour une sortie servo : **S1=S; N; 1000; 2000; 5000**

2. Si la sortie est configurée en mode imPulsionnel, un premier appui sur le bouton-poussoir associé l'active, un second appui la désactive.

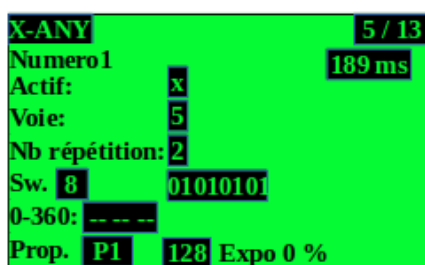
Ex :

- Pour une sortie numérique : **S2=D; P**
- Pour une sortie clignotant: **S1=B; P; 500; 500**
- Pour une sortie servo : **S2=S; P; 2000; 1000; 4000**

6.3 Configuration de l'émetteur OpenAVRc pour protocole RCUL/X-Any

Reportez vous au manuel d'**OpenAVRc** pour configurer l'instance **X-Any** avec les paramètres suivants :

1. Le N° de voie doit correspondre au N° de voie sur laquelle le décodeur **MS8-Xany V3** sera connecté sur le récepteur
2. Le nombre de répétition sera en premier lieu réglé sur 3 (dès que ça fonctionnera, il sera possible de réduire cette valeur afin d'atteindre la réactivité maximale autorisée par votre ensemble HF).
3. Configurer « Sw. » sur Sw.8 : cela va transmettre l'état de 8 contacts
4. Si le servo proportionnel sera utilisé sur **MS8-Xany V3**, sélectionner un des choix proposés par « Prop. », cela va ajouter la transmission de la valeur du proportionnelle.



6.4 Configuration de MS8-Xany V3 pour protocole RCUL/X-Any

L'émetteur **OpenAVRc** intègre nativement le protocole **RCUL/X-Any** qui permet de passer les commandes « au dessus » de n'importe quelle Interface RC de commande : PWM, CPPM, SBUS.

Comme **X-Any** est un protocole entièrement numérique, il peut passer la commande pour piloter une sortie proportionnelle (S0) en même temps que la commande des 8 sorties (S1 à S8).

Les combinaisons d'Interface RC de commande possibles sont les suivantes :

- ◆ **I=P** → **MS8-Xany V3** se connecte sur une sortie « voie » **PWM** d'un récepteur RC
- ◆ **I=C5** → **MS8-Xany V3** se connecte sur une sortie **CPPM** d'un récepteur RC et utilise la voie N°5 du train CPPM
- ◆ **I=S5** → **MS8-Xany V3** se connecte sur une sortie **SBUS** d'un récepteur RC et utilise la voie N°5 de la trame SBUS

Le protocole **RCUL/X-Any** se configure comme suit :

- ◆ **P=R; F0** → **MS8-Xany V3** utilise le Protocole **RCUL/X-Any** (**F0** : Non filtré, **F1-F3** : Filtré)

6.5 Configuration des émetteurs OpenTx et EdgeTx pour protocole binaire

Pour commander **MS8-Xany V3** à partir de 8 interrupteurs natifs (ou logiques) des émetteurs **OpenTx** et **EdgeTx**, il est nécessaire de configurer 9 mélangeurs :

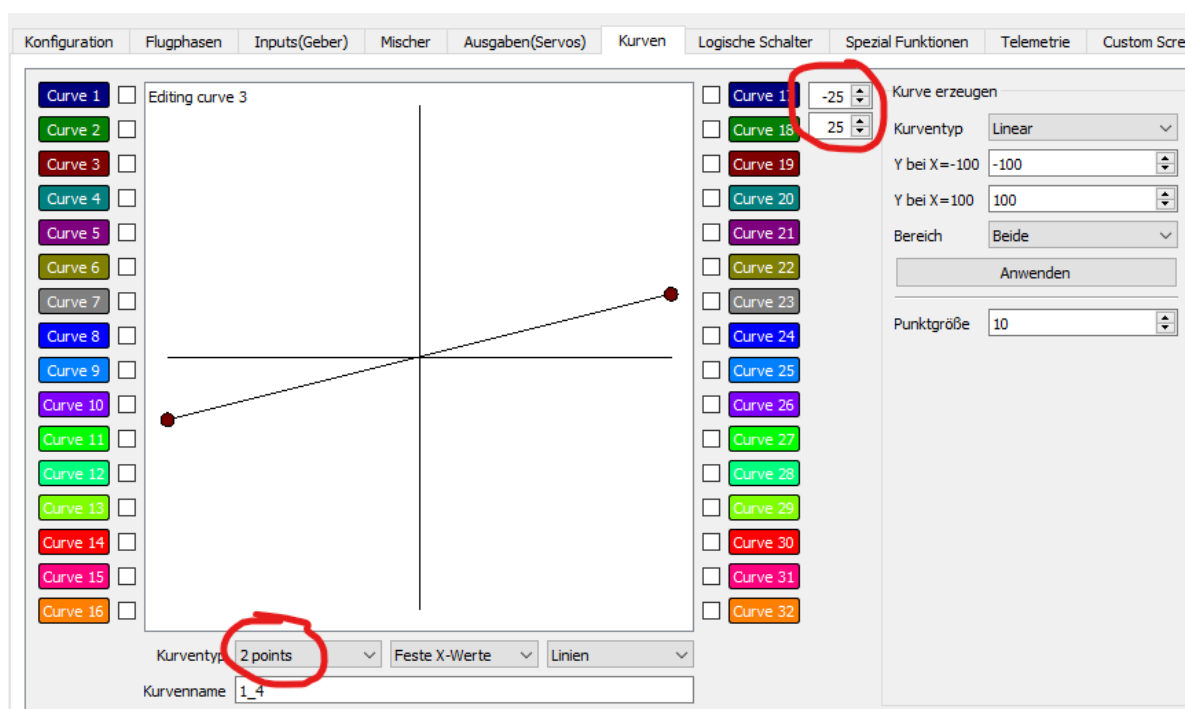
- 1 mélangeur par interrupteur → 8 mélangeurs
- 1 dernier mélangeur pour ramener les largeurs d'impulsion RC dans de la plage 1000 à 2000 µs

Chaque interrupteur représentant un bit de pondération dans le mot de 8 bits transmis sous forme de largeur d'impulsion dans la voie SBUS dédiée à **MS8-Xany V3** (voie N°5 dans l'exemple ci-dessous).

Les configurations suivantes pour **Compagnon d'OpenTx** et d'**EdgeTx** proviennent de :

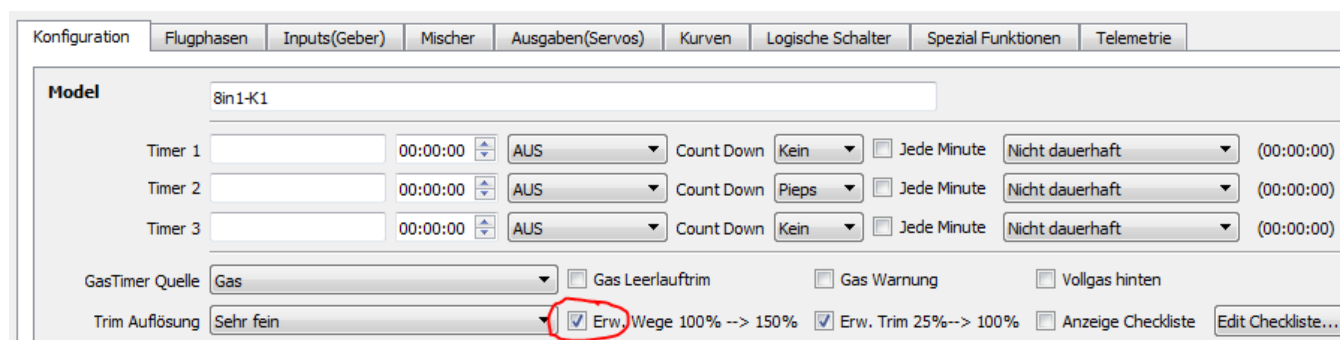
https://github.com/Tiefflieger68/SBUS-Switch/blob/main/docs/SBUS-Switch_V2.4.e.pdf

Captures d'écran d'**openTX Companion** pour la programmation de l'émetteur (**OpenTx** Version 2.4.x)



Courbe (ici Courbe 3)

Une courbe doit être définie afin de pouvoir traiter des valeurs inférieures à 1 % dans les mélangeurs.



Dans le menu "Configurations", "Chemins étendus" doit être activé. (Pas nécessaire pour le mode de compatibilité)

#	Name	Subtrim	Min	Max	Direction	Curve	PPM Center	Linear Sub
CH1		☐ GV 0,0%	☐ GV -125,0%	☐ GV 125,0%	---	---	1520us	☐

Selon le mode (Normal ou de Compatibilité), la sortie des 8 canaux utilisés dans la voie SBUS doit être ajustée en conséquence.

Différentes valeurs sont à utiliser :

	FrSky (Normal)	FrSky (Compatibilité)	FlySky (Compatibilité)
Sortie Min (%)	-125,0	-99,9	-102,4
Sortie Max (%)	+125,0	+100	+102,4
Neutre (µs)	1520	1520	1500

Konfiguration	Flugphasen	Inputs(Geber)	Mischer	Ausgaben(Servos)	Kurven	Logische Schalter	Spezial Funktionen	Tele
CH5 <pre> MAX Weight(-398%) Curve(CV3:1_4) [shift] += MAX Weight(+3%) Switch(SA↑) NoTrim Curve(CV3:1_4) [1] += MAX Weight(+6%) Switch(SB↑) NoTrim Curve(CV3:1_4) [2] += MAX Weight(+12%) Switch(SC↑) NoTrim Curve(CV3:1_4) [3] += MAX Weight(+25%) Switch(SD↑) NoTrim Curve(CV3:1_4) [4] += MAX Weight(+50%) Switch(L05) NoTrim Curve(CV3:1_4) [5] += MAX Weight(+100%) Switch(L06) NoTrim Curve(CV3:1_4) [6] += MAX Weight(+200%) Switch(L07) NoTrim Curve(CV3:1_4) [7] += MAX Weight(+400%) Switch(L08) NoTrim Curve(CV3:1_4) [8] </pre>								

Mélangeur:

Le premier mélangeur est le mélangeur commun. Aucun interrupteur ne lui est affecté.

Les autres mélangeurs correspondent aux sorties 1 à 8.

Un commutateur physique ou logique doit être attribué dans chaque cas. La sortie de commutation correspondante est alors commutée avec cet interrupteur.

6.6 Configuration de MS8-Xany V3 pour OpenTx et EdgeTx pour protocole binaire

Sauf **EKMFA**, la seule interface RC supportée avec **OpenTx** et **EdgeTx** est l'interface SBUS avec le protocole « Binaire » :

- ◆ **I=S5** → **MS8-Xany V3** se connecte sur une sortie **SBUS** d'un récepteur RC et utilise la voie N°5 de la trame SBUS.

Note : Il peut être nécessaire d'ajouter le paramètre optionnel **;-** ou **;** à la fin de la commande (cf §COMPENSATION DE L'ERREUR DE DEBIT SERIE)

- ◆ **P=B** → Protocole « Binaire » en SBUS

Le protocole Binaire en SBUS permet l'utilisation des interrupteurs physiques et logiques, ce qui est beaucoup plus pratique que la commande par coups de manche de **EKMFA**.

6.7 Configuration de MS8-Xany V3 pour protocole Robbe-Futaba

MS8-Xany V3 est en mesure de se substituer à un décodeur MultiSwitch 8 sorties N° F1513 de **Robbe-Futaba**.

Si un ensemble RC **Robbe-Futaba** fonctionne avec le décodeur MultiSwitch 8 sorties N° F1513, il fonctionnera également avec **MS8-Xany V3**.

Les 2 seuls modes supportés avec **Robbe-Futaba** sont les modes d'interface RC PWM et CPPM avec le protocole «Futaba» :

- ◆ **I=P** → **MS8-Xany V3** se connecte sur une sortie « voie » **PWM** d'un récepteur RC
- ◆ **I=C7** → **MS8-Xany V3** se connecte sur une sortie **CPPM** d'un récepteur RC et utilise la voie N°7 du train **CPPM**
- ◆ **P=F** → Le **Protocole «Futaba»** en PWM ou en CPPM

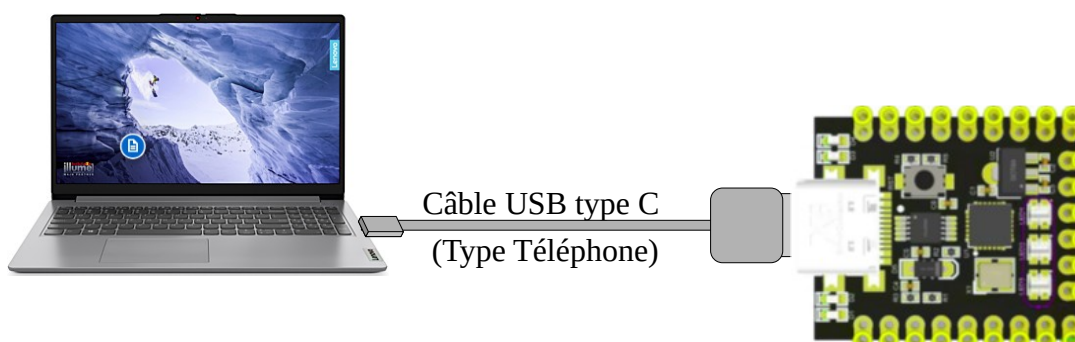
7 MODE AVANCÉ/COMMANDE DE SERVOS

Le décodeur **MS8-Xany V3** dispose d'un accès USB pour les configurations avancées.

C'est cet accès série USB de **MS8-Xany V3** qui va permettre le paramétrage, par exemple, de la sélection de l'interface RC, du protocole utilisé et l'utilisation de servos connectés aux sorties **S1** à **S8**. La liste des commandes supportées est détaillée en page suivante.

7.1 Utilisation du port série de configuration de MS8-Xany V3

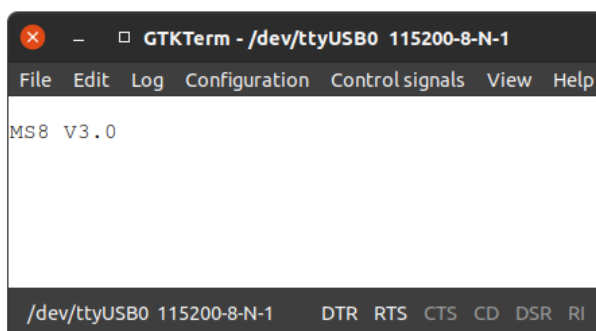
Pour accéder au port série USB de configuration de **MS8-Xany V3**, il faut un simple câble USB type C (type câble USB de smartphone) :



1. Connecter le câble USB type C entre un PC le port USB du **Mini Nano V3**
2. Sur le PC, ouvrir un Terminal série, par exemple, PuTTY, TeraTerm, HyperTerminal, GtkTerm, ou encore CoolTerm avec les paramètres suivants : 115200 bauds, 8 bits de données, 1 bit de stop, pas de parité.

Selon le Terminal série, il peut être nécessaire d'activer les retours à la ligne automatique sur réception de CR/LF pour avoir un bon affichage.

3. Appuyer sur la touche « Entrée » de votre clavier, le message « MS8 V3.X » doit apparaître sur le Terminal série comme illustré ci-dessous.



Exemple de connexion avec le Terminal GtkTerm sous Linux

Note :

Il est possible de configurer **MS8-Xany V3** pendant que celui-ci est connecté au récepteur RC.

7.2 L'aide intégrée

Taper « H? » puis, Entrée pour obtenir l'aide intégrée :

```
GTKTerm - /dev/ttyUSB0 115200-8-N-1
File Edit Log Configuration Controlsignals View Help
MS8 V3.0
H?
H? -> Returns this help
<--' -> If Enter is sent, MS8 Vx.y stays in Terminal Mode and
      failsafe is disabled (nice, for config and test)
      Additionally, this command triggers the display of the
      Welcome Message "MS8 Vx.y" in the Terminal
C? -> Display the full Configuration with status
      (all the commands listed below)
C=F -> Set the Factory default parameters (I=P P=E SR=N Sx=D;N)
T=CRLF/CR -> Set the Line Terminator to CRLF or to CR
T? -> Return the configured Line Terminator
I=Interf[Ch[;-/+]] -> Set the RC Interface (P for PWM, C for CPPM, S for SBUS,
      X for SRXL, D for SUMD, I for IBUS, F for CRSF), Ch is the
      channel N°, optional - or + tunes the soft serial data rate
I? -> Return the configured Interface
P=MsProto[;Fl] -> Set the Multi-Switch Protocol (MsProto: R for RCUL with
      l=Filter level (from 0 to 3), F for MS8 Futaba, B for Binary
      (for OTX and ETX), P for Push-Button and E for EKMFA)
P? -> Return the configured Multi-Switch Protocol
S0=XXXX -> If 988 <= XXXX <= 2008, set Prop Servo to the XXXX position
      in us. If 0 <= XXX <= 255, set Prop Value Command
SR=N/Y -> Apply reverse (Y) or no reverse (N) for Prop Servo (S0)
Sx=D;M -> Sx is a Digital output in mode M (M=Normal or Pulsed)
      (x is the id of the output from 1 to 8, x = 0 is reserved for
      the prop Servo)
Sx=S;M;PosA;PosB;A2BDur[;B2ADur] -> Sx is a Servo output in mode M (M=Normal or
      Pulsed). PosA us when Cmd=0, PosB us when Cmd=1, Duration
      between Pos A & B are A2BDur & B2ADur (64000 ms Max)
Sx=B;M;OnDur[;OffDur] -> Sx is a Blink output in mode M (M=Normal or P=Pulsed)
      OnDur and OffDur are On & Off duration in ms (64000 ms Max)
Sx=0/1 -> Simulate a Cmd=0/1 command for Sx (Nice for test without RC)
Sx? -> Return Sx=D;M:C if output x is Digital, or
      Sx=S;M;PosA;PosB;A2BDur;B2ADur:C if output x is a Servo, or
      S0=XXX:CCC for Prop Servo (C is the current Command status)
S8=A;N/I -> Additionally to Digital and Servo Type, S8 can be configured
      as pwm Analog output with Normal(N) or Inverted (I) polarity
S8=xxx[%] -> When S8 configured as Analog, S8=xxx sets the Cmd Prop Value
      (0-255) or S8=xxx% sets the percentage of pwm (0%-100%)
S8? -> When S8 configured as Analog, the answer to the S8? command
      is S8=A;N/I:PwmPerCent%:PropValue (eg: S8=A;I:100%:000)
B=C -> Start the Button Calibration process
B? -> Return the pulse width (in us) associated to each of the
      8 Buttons: B=xxxx,xxxx,xxxx,xxxx,xxxx,xxxx,xxxx,xxxx
D=DebugLevel -> Set the Debug Level: 1 -> Pulse Width, 2 -> Cmd Change
D? -> Return the Debug Level
Q -> Quit the Terminal Mode and switch to RC Mode.
      To switch back to Terminal Mode, simply hit Enter

/dev/ttyUSB0 115200-8-N-1 DTR RTS CTS CD DSR RI
```

7.3 Les messages de commande de MS8-Xany V3

La liste des messages supportés par **MS8-Xany V3** est donnée dans la table suivante :

← Commande/ → Réponse	Action	Remarque
← Enter → MS8 V3.x	Retourne le nom et la version du Firmware de MS8-Xany V3	
← H? → L'aide intégrée	Retourne l'aide intégrée	Voir §L'aide intégrée
← T? → T=CRLF/CR	Retourne le type courant de Terminateur de ligne pour le Terminal Série Termineurs de ligne possibles : - CRLF: CaRriage Return + Line Feed (Retour Charriot + Passage à la ligne). - CR: CaRriage Return (Retour Charriot).	
← T=CRLF/CR → T	Définit le Terminateur de ligne pour le Terminal Série	Mettre CRLF pour le Terminal Serial Console de l'IDE Arduino
← I? → I=Interface[v][;-/+]	Retourne l'Interface RC utilisée par MS8-Xany V3 . Interfaces possibles : - P: PWM. MS8-Xany V3 est connecté sur la voie PWM du récepteur, v n'est pas nécessaire. - Cv: CPPM voie v. MS8-Xany V3 est connecté sur la sortie CPPM du récepteur et utilise la voie v. - Sv: SBUS voie v. MS8-Xany V3 est connecté sur la sortie SBUS du récepteur et utilise la voie v. - Xv: SRXL voie v. MS8-Xany V3 est connecté sur la sortie SRXL du récepteur et utilise la voie v. - Dv: SUMD voie v. MS8-Xany V3 est connecté sur la sortie SUMD du récepteur et utilise la voie v. - Sv: IBUS voie v. MS8-Xany V3 est connecté sur la sortie IBUS du récepteur et utilise la voie v. - Fv: CRSF voie v. MS8-Xany V3 est connecté sur la sortie CRSF du récepteur et utilise la voie v.	La voie v doit être dans la plage 1 à 16. Le paramètre optionnel ;- ou ;+ sert à corriger le débit 100000 bauds (SBUS) ou 115200 Bauds (SRXL, SUMD, IBUS, CRSF) selon le pourcentage d'erreur sur le débit de la sortie série du récepteur (dépend de sa fréquence d'horloge interne). La commande de debug D=1 (cf plus bas) permet de vérifier le meilleur réglage. Exemple : I=S5 ;- → Tolérance basse I=S5 → Tolérance moyenne I=S5 ;+ → Tolérance haute

← I=Interface[v] → I	Définit l'Interface RC utilisée par MS8-Xany V3 . Idem, ci-dessus.	
← P? → P=R;FI ou → P=F/B/E/P	Retourne le Protocole utilisé sur l'interface RC : - R;FI : Protocole RCUL/X-Any. Il faut préciser le niveau I de Filtrage. - F : Protocole Futaba MultiSwitch 8 - B : Binaire (Valeur 8 bits de 0 à 255 codée dans l'impulsion RC) - E : Protocole EKMFA (le nombre de coup de manche active/désactive une sortie) - P : Protocole Boutons-Poussoirs	- Utiliser le protocole RCUL/X-Any pour OpenAVRc Le paramètre I de filtrage vaut : - F0 : Non filtré (niveau 0) - F1 à F3 : Filtré (niveau 1 à 3) Dans la plupart des cas, ce paramètre doit être positionné à F0 (Non filtré). Un récepteur RC effectue un filtrage quand il considère une grande variation d'impulsion RC comme probablement anormale. Par exemple, si l'impulsion RC passe brutalement de 1000µs à 2000µs, l'impulsion en sortie du récepteur va passer de 1000µs à 1500µs avant d'atteindre 2000µs. C'est le comportement des récepteurs dits « DSP » ou encore des récepteurs PCM Futaba. - Utiliser le protocole Futaba pour remplacement d'un décodeur MultiSwitch 8 sorties N°F1513 de Robbe-Futaba - Utiliser le protocole Binaire pour OpenTx et pour EdgeTx - Pour tous les autres ensembles RC, utiliser les protocoles Boutons-Poussoirs ou EKMFA
← P=R;FI/F/B/E/P → P	Définit le Protocole utilisé sur l'interface RC (idem ci-dessus)	
← SR? → SR=Y/N	Retourne l'inversion de servo pour la sortie S0. Y=Inversé, N=Non inversé	
← SR=Y/N → SR	Définit l'inversion de servo pour la sortie S0. Y=Inversé, N=Non inversé	
← S0? → S0=Pos:OffsetPas4us	Retourne la position courante en µs et la commande OffsetPas4us qui est le nombre de pas de 4µs (valeur entre 0 et 255) à ajouter à 988 pour avoir la largeur d'impulsion en µs pour le servo proportionnel	Si SR=N : Largeur d'impulsion(us) = 988 + (OffsetPas4us x 4) Si SR=Y : Largeur d'impulsion(us) = 988 + [(255 - OffsetPas4us) x 4]
← S0=Val → S0	Si $988 \leq \text{Val} \leq 2008$: définit la position en µs pour le servo proportionnel Si $0 \leq \text{Val} \leq 255$: définit la valeur de la commande 8 bits telle qu'elle serait envoyée par le protocole RCUL	Renvoie ERR, si valeur non comprise entre 988 et 2008 ou si non comprise entre 0 et 255.

← Sx? → Sx=D;M:C ou : → Sx=B;M;OnDur;OffDur:C ou : → Sx=S;M;PosA;PosB;A2BDur;B2ADur:C	Retourne la configuration de la sortie N°x ainsi que l'état « C » de la Commande courante associée (0 ou 1) - Si la sortie est configurée en sortie numérique MultiSwitch, la réponse est : Sx=D;M:C D=Digital (Numérique) M=Mode de commande (N : Normal, P : ImPulsionnel) - Si la sortie est configurée en sortie clignotant, la réponse est : Sx=B;M;OnDur;OffDur:C avec B=Blink (Clignotant) M=Mode de commande (N : Normal, OnDur : Durée à l'état ON en ms OffDur : Durée à l'état OFF en ms - Si la sortie est configurée en sortie Servo, la réponse est : Sx=S;M;PosA;PosB;A2BDur;B2ADur]:C avec S=Servo M=Mode de commande (N : Normal, P : ImPulsionnel) PosA=la position en µs pour une commande à 0, PosB=la position en µs pour une commande à 1, et A2BDur=la durée du mouvement du servo entre PosA et PosB B2ADur=la durée du mouvement du servo entre PosB et PosA	Renvoie ERR, si - Valeur x non comprise entre 0 et 8 - PosA ou PosB < 600 - PosA ou PosB > 2400 Exemple de réponses : S1=D;N:0 S2=D;P:1 S3=B;N:0250;0750:0 S3=B;P:0500;0500:1 S3=S;N:1000;2000;5000:0 S4=S;P;2300;600;8500;2000:1
← Sx=D;M → Sx	Définit la sortie x comme étant de type numérique (Digital) et commandée en mode M (M=N : Mode Normal, M=P : Mode imPulsionnel)	Exemple de commandes : S1=D;N S2=D;P
← Sx=B;M;OnDur;OffDur] → Sx	Définit la sortie x comme étant de type clignotant (Blink) et commandée en mode M (M=N : Mode Normal, M=P : Mode imPulsionnel). OnDur est la Durée en ms à l'état ON et OffDur est la Durée en ms à l'état OFF	Exemple de commandes : S1=B;N;250;750 S2=B;P;500;500 Si la valeur optionnelle OffDur n'est pas fournie, elle prend automatiquement la même valeur que OnDur.
← Sx=S;M;PosA;PosB;A2BDur;B2ADur] → Sx	Définit la sortie x comme étant de type Servo commandée en mode M (M=N : Mode Normal, M=P : Mode imPulsionnel) avec une position à PosA µs pour une commande à 0, une position à PosB µs pour une commande à 1, et une durée de mouvement à A2BDur ms pour la	La valeur «A2BDur» et «B2ADur» en ms sont recalculées en interne par MS8-Xany V3 en tenant compte des différentes résolutions/limitations et peuvent être légèrement différentes à la relecture.

	direction A → B et optionnellement pour une durée de mouvement B2ADur ms pour la direction B → A.	Si la valeur optionnelle B2ADur n'est pas fournie, elle prend automatiquement la même valeur que A2BDur.
← Sx=C → Sx	Si x est compris entre 1 et 8, « C » définit la Commande (0 ou 1) pour la sortie x, que celle-ci soit de type Digital, Blink (clignotant) ou Servo	Très pratique pour les tests par l'accès série sans RC.
← S8=A;N/I → Sx	S8, en plus de pouvoir fonctionner en mode Digital, peut être configurée en une sortie Analogique en PWM. S8=A;N : Analogique polarité Normale, S8=A;I : Analogique polarité Inversée	Pour pouvoir exploiter S8 en sortie Analogique, il faut utiliser le protocole RCUL, le seul à pouvoir passer des valeurs proportionnelles.
← B=C → BPx:	Entre dans la mode de calibration des boutons-poussoirs	
← B? → B=xxxx;xxxx;...;xxxx	Retourne les valeurs de calibration (µs) des 8 boutons-poussoirs	Les valeurs vont environ de 1000 à 2000µs
← C? → La conf complète avec affichage de l'état de la commande associée	Retourne la Configuration complète de MS8-Xany V3	L'état des commandes est présent en fin de ligne après le ':'
← A? → La conf complète	Retourne la Configuration complète pour archivage	Ne contient pas les états en fin de ligne après le ':'
← D=NiveauDebug → D	Définit le niveau de debug : D=0 : désactive le debug D=1 : les largeurs d'impulsion reçues sur l'entrée RC sont affichées avec la durée en ms entre chaque impulsion. D=2 : les changements de commandes reçus sont affichés	
← D? → D=NiveauDebug	Retourne le niveau de debug courant	Après une mise sous tension de MS8-Xany V3 , le Debug est à 0.

Note :

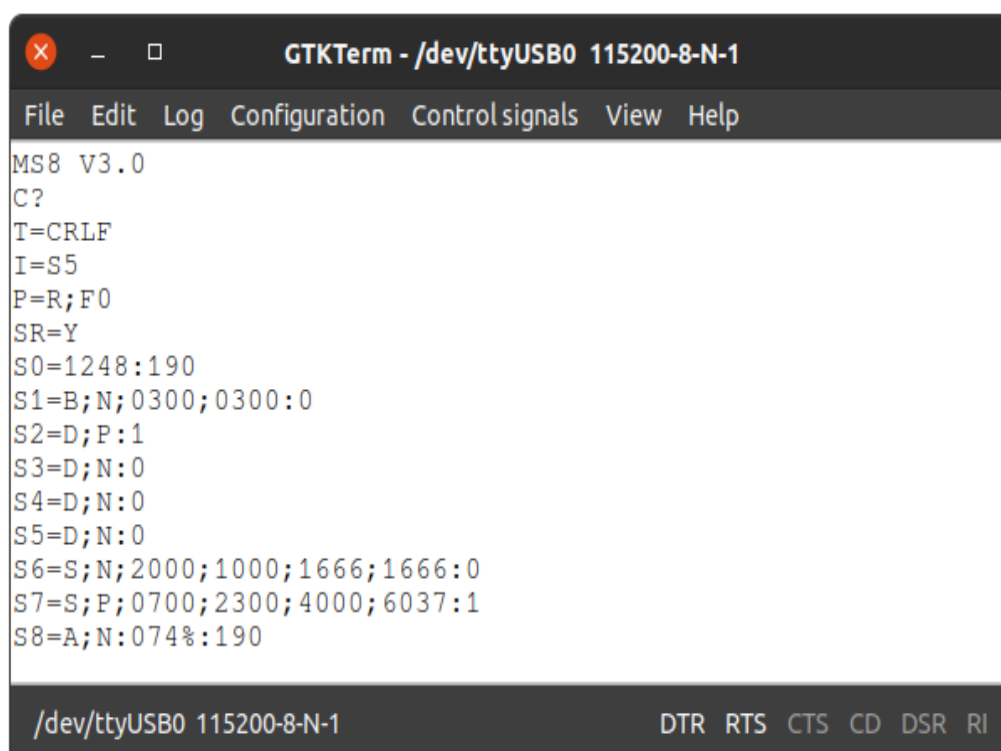
L'aide intégrée (commande H?) donne les mêmes informations quant aux commandes possibles.

7.4 Exemple de configuration réelle

Dans l'exemple ci-dessous :

- **MS8-Xany V3** est connecté sur l'interface SBUS d'un récepteur et utilise la voie N°5.
- Le protocole utilisé sur SBUS est RCUL/X-Any sans filtrage (F0).
- La sortie proportionnelle (S0) est inversée et commandée par la valeur proportionnelle transmise dans le message RCUL.
- La largeur d'impulsion pour le servo/ESC connecté sur la sortie proportionnelle S0 vaut :
$$988 + [(255 - \mathbf{190}) \times 4] = \mathbf{1248} \mu\text{s} \quad ((255 - \mathbf{190}) \text{ et non directement } \mathbf{190}, \text{ car inversé})$$

→ la commande (= le nombre de pas de 4 μs à ajouter à 988 μs) est de **190** pour la largeur d'impulsion de **1248 μs**
- La sortie S1 est de type Blink (Clignotant) commandée en mode Normal (ON : 300 ms, OFF : 300 ms). Sa commande vaut 0.
- Les sorties S2, S3, S4 et S5 sont de type Digital (MultiSwitch), leurs commandes valent respectivement 1, 0, 0 et 0. La sortie S2 est commandée en imPulsionnel.
- Les sorties S6, S7 sont de type Servo, leurs commandes valent respectivement 0 et 1.
La sortie 7 est commandée en imPulsionnel avec des durées de mouvement différentes.
- La sortie S8 est de type Analogique avec polarité Non-inversée (PWM commandé par la valeur proportionnelle transmise dans le message RCUL : **74%** de PWM pour Prop=**190**).

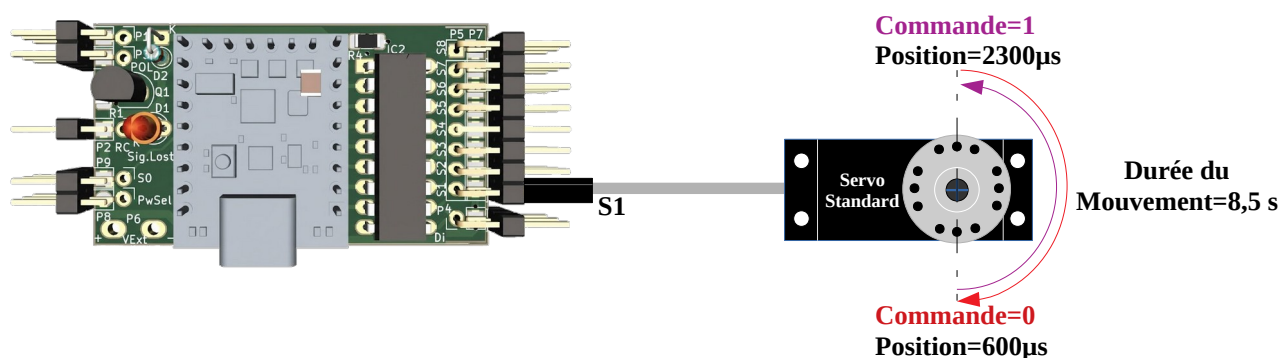


```
GTKTerm - /dev/ttyUSB0 115200-8-N-1
File Edit Log Configuration Control signals View Help
MS8 V3.0
C?
T=CRLF
I=S5
P=R;F0
SR=Y
S0=1248:190
S1=B;N;0300;0300:0
S2=D;P:1
S3=D;N:0
S4=D;N:0
S5=D;N:0
S6=S;N;2000;1000;1666;1666:0
S7=S;P;0700;2300;4000;6037:1
S8=A;N:074%;190
/dev/ttyUSB0 115200-8-N-1 DTR RTS CTS CD DSR RI
```

1. Mouvement de servo dans le sens contraire des aiguilles d'une montre

Si le contact N°1 côté émetteur est fermé (**Commande=1**), l'impulsion du servo connecté à la sortie N°1 va de 600 μ s à 2300 μ s (soit un mouvement d'environ 180°) en 8,5 secondes, et va de 2300 μ s à 600 μ s (soit un mouvement d'environ 180°) en 8,5 secondes si le contact N°1 côté émetteur est ouvert (**Commande=0**).

Ex : **S1=S;N;600;2300;8500**

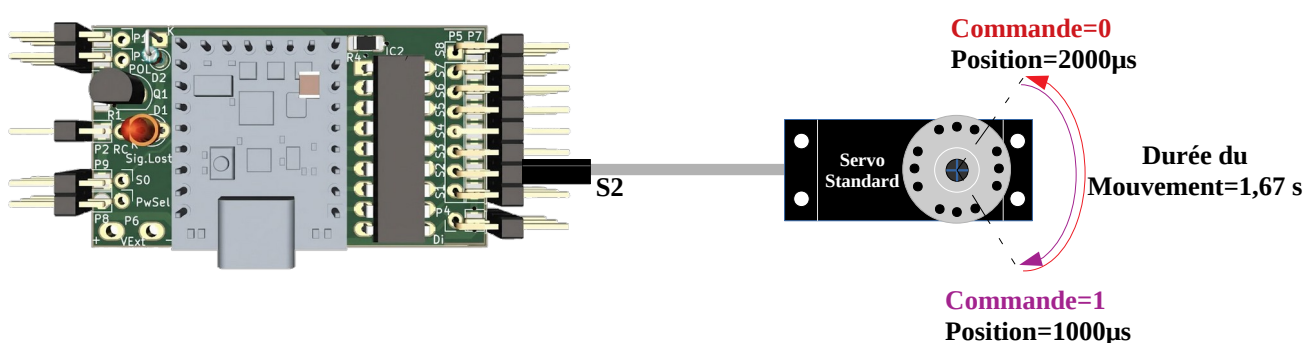


2. Mouvement de servo dans le sens des aiguilles d'une montre

Avec **MS8-Xany V3**, il est possible d'avoir un mouvement dans le sens opposé : il suffit de permuter les positions extrêmes (Pos0 et Pos1).

Si le contact N°2 côté émetteur est fermé (**Commande=1**), l'impulsion du servo connecté à la sortie N°2 va de 2000 μ s à 1000 μ s (soit un mouvement d'environ 100°) en 1,67 secondes si la commande côté émetteur vaut 1, et de 1000 μ s à 2000 μ s (soit un mouvement d'environ 100°) en 1,67 secondes si le contact N°2 côté émetteur est ouvert (**Commande=0**).

Ex : **S2=S;N;2000;1000;1666**



7.5 Différence entre mode Normal et mode imPulsionnel

Que la sortie soit configurée en type **Digital** (Multiswitch) ou en type **Servo**, il faut définir le mode commande :

- **Sx=D;M** (avec M valant **N** pour **Normal** ou **P** pour im**P**ulsionnel)
- **Sx=S;M;1000;2000;5000** (avec M valant **N** pour **Normal** ou **P** pour im**P**ulsionnel)

Supposons que l'on ait la configuration suivante et que, côté émetteur, S1 et S2 soient commandés par des contacts de boutons-poussoirs :

S1=D;N pour piloter une corne de brume

S2=D;P pour piloter une lumière

Dans les 2 cas, les sorties sont de type **Digital** (Multiswitch), mais **S1** est pilotée en mode **Normal**, tandis que **S2** est pilotée en mode im**P**ulsionnel.

1. Cas du mode Normal :

Tant que l'on appuie sur le contact C1, la sortie S1 est à 1 et la corne de brume fonctionne.

Dès que l'on relâche le contact C1, la sortie S1 passe à 0 et la corne de brume s'arrête.

En résumé, la sortie S1 suit l'état du contact C1.

2. Cas du mode imPulsionnel :

Tant que l'on appuie sur le contact C2, la sortie S2 est à 1 et la lumière fonctionne.

Dès que l'on relâche le contact C2, la sortie S2 reste à 1 et la lumière continue à fonctionner.

Pour éteindre la lumière, il faut appuyer une nouvelle fois sur le contact C2.

En résumé, une impulsion sur le contact C2 active la sortie S2, une nouvelle impulsion sur le contact C2 désactive la sortie S2 et ainsi de suite.

Notes :

1. Si la sortie est de type **Servo** en mode im**P**ulsionnel, une impulsion sur le bouton-poussoir, provoque le mouvement du servo jusqu'à la position N°1 et une seconde impulsion provoque le mouvement du servo jusqu'à la position N°2.

Ex : **S3=S;P;1000;2000;5000**

2. En protocole **EKMFA**, même si le mode de la sortie est configuré en **Normal**, elle est toujours traitée comme étant configurée en im**P**ulsionnel.

8 LES POSSIBILITES DE DEBUG/DIAGNOSTIC

MS8-Xany V3 offre quelques possibilités de debug/diagnostic afin d'identifier un problème ou tout simplement pour vérifier la bonne configuration. Pour utiliser ces commandes de debug/diagnostic, un signal RC doit être présent à l'entrée RC de **MS8-Xany V3**.

8.1 Vérification de la bonne réception du signal RC

- **D=1**

```
GTKTerm - /dev/ttyUSB0 115200-8-N-1
File Edit Log Configuration Control signals View Help
MS8 V3.0
D=1
D
T=136:1492us
T=015:1492us
T=021:1492us
T=018:1492us
T=017:1492us
T=019:1492us
T=017:1496us
/dev/ttyUSB0 115200-8-N-1 DTR RTS CTS CD DSR RI
```

```
GTKTerm - /dev/ttyUSB0 115200-8-N-1
File Edit Log Configuration Control signals View Help
MS8 V3.0
D=1
D
T=089:1864us->F
T=009:1864us->F
T=009:1023us->0
T=008:1023us->0
T=011:1920us->R
T=017:1135us->2
T=008:1135us->2
/dev/ttyUSB0 115200-8-N-1 DTR RTS CTS CD DSR RI
```

La largeur des impulsions reçues (en μs) ainsi que le temps entre impulsions (en ms) sont affichés. Les symboles transportant le message RCUL/X-Any sont affichés uniquement si le protocole est réglé sur RCUL/X-Any (figure de droite, ci-dessus). Envoyer la commande D=0 pour arrêter l'affichage.

8.2 Vérification de la bonne prise en compte des commandes

- **D=2**

```
GTKTerm - /dev/ttyUSB0 115200-8-N-1
File Edit Log Configuration Control signals View Help
MS8 V3.0
S: 1 2 3 4 5 6 7 8 P
CmdV-> 1 0 0 0 0 0 0 0 1 1
      2 2
      7 7
S: 1 2 3 4 5 6 7 8 P
CmdV-> 0 0 0 0 0 0 0 0 1 1
      2 2
      7 7
/dev/ttyUSB0 115200-8-N-1 DTR RTS CTS CD DSR RI
```

Diagram annotations: Red arrows labeled S1 point to the 'S' column header and the 'S' row of the data table. Red arrows labeled CmdV point to the 'CmdV' row header and the 'CmdV' row of the data table.

Ici, la Valeur de la Commande de **S1** est passée à 1, puis est repassée à 0 et la Valeur de Commande Proportionnelle pilotant S8 et S0 (P) vaut 127.

9 COMPENSATION DE L'ERREUR DE DEBIT SERIE

Les protocoles SBUS, SRXL, SUMD, IBUS et CRSF sont dits « série » car ils sont accessibles depuis une sortie série de type UART d'un récepteur RC.

La tolérance sur le débit série de cet UART dépend de la fréquence d'horloge (le Quartz) du microcontrôleur présent dans le récepteur RC.

Cette erreur peut atteindre jusqu'à 3 %, ce qui crée des problèmes de « cadrage » (« framing » en anglais) et conduit à la corruption des caractères transportés.

MS8-Xany V3 est sensible à cette tolérance car la réception des caractères à 100000 Bauds (SBUS) et à 115200 Bauds (SRXL, SUMD, IBUS et CRSF) est réalisée de manière purement logicielle.

C'est pourquoi, pour les protocoles RC série (SBUS, SRXL, SUMD, IBUS et CRSF), il peut être nécessaire de compenser ce pourcentage d'erreur sur le débit série généré par le récepteur.

MS8-Xany V3 propose un paramètre optionnel pour la commande d'Interface permettant de compenser cette erreur de débit.

Exemple de compensation de la tolérance série en CRSF :

I=F5 ; - → Tolérance basse
I=F5 → Tolérance moyenne (pas de compensation)
I=F5 ; + → Tolérance haute

Quand **MS8-Xany V3** est connecté à la sortie série du récepteur RC, la commande de debug D=1 permet de vérifier laquelle de ces 3 combinaisons conduit au maximum de réception (surveiller la valeur T=xxx, il doit y avoir le minimum de valeurs xxx supérieures à 030).

10 TABLES DES PROTOCOLES ET DES INTERFACES RC SUPPORTÉS

10.1 Tout ensemble RC

		Protocole de commande				
		EKMFA	Boutons-Poussoirs	RCUL/X-Any	Binaire	Futaba
Interface RC	PWM	✓	✓	✗	✗	✗
	CPPM	✓	✓	✗	✗	✗
	SBUS	✓	✓	✗	✗	✗
	SRXL	✓	✓	✗	✗	✗
	SUMD	✓	✓	✗	✗	✗
	IBUS	✓	✓	✗	✗	✗
	CRSF	✓	✓	✗	✗	✗

10.2 Ensemble OpenAVRc

		Protocole de commande				
		EKMFA	Boutons-Poussoirs	RCUL/X-Any	Binaire	Futaba
Interface RC	PWM	✓	✓	✓	✗	✗
	CPPM	✓	✓	✓	✗	✗
	SBUS	✓	✓	✓	?	✗
	SRXL	✓	✓	✓	✗	✗
	SUMD	✓	✓	✓	✗	✗
	IBUS	✓	✓	✓	✗	✗
	CRSF	✓	✓	✓	✗	✗

10.3 Ensembles OpenTx/EdgeTx

		Protocole de commande				
		EKMFA	Boutons-Poussoirs	RCUL/X-Any	Binaire	Futaba
Interface RC	PWM	✓	✓	✗	✗	✗
	CPPM	✓	✓	✗	✗	✗
	SBUS	✓	✓	✗	✓	✗
	SRXL	✓	✓	✗	✗	✗
	SUMD	✓	✓	✗	✗	✗
	IBUS	✓	✓	✗	✗	✗
	CRSF	✓	✓	✗	✗	✗

10.4 Ensemble Futaba supportant le MultiSwitch 8 sorties Futaba N° F1513

		Protocole de commande				
		EKMFA	Boutons-Poussoirs	RCUL/X-Any	Binaire	Futaba
Interface RC	PWM	✓	✓	✗	✗	✓
	CPPM	✓	✓	✗	✗	✓
	SBUS	✓	✓	✗	✗	✗
	SRXL	✓	✓	✗	✗	✗
	SUMD	✓	✓	✗	✗	✗
	IBUS	✓	✓	✗	✗	✗
	CRSF	✓	✓	✗	✗	✗